

NUTRITION ASSISTANT

Project Documentation

A MERN Stack-Based Personalized Nutrition & Diet Management Solution

Technology Stack: HTML · CSS · JavaScript · React.js · Node.js · Express.js · MongoDB

Table of Contents

1. Project Overview
2. Problem Statement
3. Requirement Gathering
4. System Architecture
5. Functional Scope / Key Features
6. Stakeholder Mapping & Roles
7. User Flow
8. Execution Roadmap
9. System Requirements
10. Data Architecture & Object Model
11. Application Structure (Backend)
12. Testing
13. Maintenance and Troubleshooting
14. Learning Outcomes
15. Output of the Project
16. Conclusion

1. Project Overview

In today's fast-paced world, maintaining a balanced diet can be challenging. The Nutrition Assistant App is designed to simplify this journey by offering personalized nutritional guidance and support. Whether the goal is improving eating habits, managing a health condition, or achieving fitness targets, the application serves as a comprehensive tool for better health and wellness.

The system leverages technology to provide meal planning, dietary tracking, personalized recommendations, and access to educational resources. By incorporating an extensive food database, the app allows users to log meals, monitor calorie and nutrient intake, and gain insight into the nutritional value of the foods they consume. Planned extensions include recipe suggestions, grocery assistance, and interactive features for connecting with nutrition professionals.

The Nutrition Assistant is built to empower individuals to make informed lifestyle choices, and is suited to managing weight, enhancing athletic performance, accommodating dietary restrictions, or simply adopting healthier habits.

Disclaimer: *The application is intended to support, not replace, professional medical or dietary advice. Users are encouraged to consult qualified healthcare providers for personalized recommendations.*

2. Problem Statement

Individuals trying to eat healthily often struggle with fragmented tools: food diaries kept on paper or in generic notes apps, no easy way to see the nutritional composition of what they eat, and no structured way to plan or monitor progress toward a diet goal. Without a centralized system, users find it difficult to track calorie and nutrient intake accurately, stay consistent with a diet plan, or receive guidance tailored to their own profile (weight, height, gender, and activity level).

A unified, easy-to-use Nutrition Assistant application was required — one that lets users log food consumption, build and follow diet plans, view nutritional breakdowns, and receive personalized recommendations, all from a single platform.

3. Requirement Gathering

Key requirements identified for the system:

- User registration and profile management, capturing username, email, age, weight, height, gender, and activity level.
- A searchable food database with calorie, carbohydrate, protein, and fat information for each item.
- Food consumption logging linked to a user account, with quantity/serving size and date.
- Diet plan creation with defined start and end dates, goals, and preferences.
- Nutritional analysis and reporting for meals and diet plans.
- Personalized recommendations based on user profile, goals, and logged data.
- Secure, authenticated access to user-specific data.
- A RESTful backend capable of scaling as the food database and user base grow.

4. System Architecture

The Nutrition Assistant system is divided into three main components: User Interface, Backend Services, and External Services.

- **User Interface:** Represents the interface through which users interact with the system — registration, food logging, diet plans, and reports.
- **Backend Services:** Includes an API for handling user requests and responses, a Database for storing and retrieving data, and a Nutrition Functions module that provides nutrition-related calculations and logic.
- **External Services:** Represents external APIs the system interacts with, such as third-party nutrition/food databases used to enrich the internal food catalog.

System Architecture

Figure 1: Nutrition Assistant — System Architecture

Requests flow from the User Interface to the API, which delegates nutrition-specific logic to the Nutrition Functions module. Both the API and Nutrition Functions read from and write to the Database, and the Nutrition Functions module can call out to External APIs when additional food or nutrition data is required.

5. Functional Scope / Key Features

User Registration and Profile Management

- Users can register an account by providing their username, email, age, weight, height, gender, and activity level.
- Users can manage their profile information, such as updating their personal details.

Food Consumption Tracking

- Users can track the food they consume by associating food items with their account.
- The application provides a database of food items with nutritional information (calories, carbohydrates, proteins, fats).
- Users can search and select food items they consume and record the quantity or serving size.

Diet Plan Creation and Monitoring

- Users can create personalized diet plans based on their goals, preferences, and nutritional requirements.
- Diet plans are associated with a specific user and have a start and end date.
- The application can recommend or suggest diet plans based on user information and goals.
- Users can monitor their adherence to the diet plan and track their progress.

Nutritional Information and Analysis

- The application provides comprehensive nutritional information for food items, meals, and diet plans.
- Users can view total calories, carbohydrates, proteins, and fats for their meals and diet plans.
- Nutritional analysis helps users make informed decisions about their food choices and monitor intake.

Recommendations and Guidance

- Based on user profiles, goals, and dietary preferences, the application provides personalized recommendations for food items, meals, and diet plans.
- Users receive guidance and tips on achieving a balanced diet, managing portion sizes, and making healthier choices.

Reporting and Progress Tracking

- Users can generate reports and visualizations to track nutritional intake, progress towards goals, and adherence to diet plans.
- The application provides insights and feedback to help users make adjustments and stay on track.

6. Stakeholder Mapping & Roles

- **Customer / End User:** The primary end-user of the application, responsible for registration, logging food, managing diet plans, and acting on recommendations.
- **System Administrator / Developer:** Maintains the food and nutrition database, monitors data accuracy, and manages backend infrastructure, security, and deployments.
- **Nutrition Professional (planned):** A future-facing role — nutritionists or dietitians who can review user data and refine recommendation logic through the app's interactive features.

Customer Responsibilities

- **Registration & Profile Management:** Responsible for creating an account by providing accurate and relevant information during registration.
 - Manages profile information, ensuring personal details such as age, weight, height, and activity level stay up to date.
- **Inputting Food Consumption:** Responsible for tracking and inputting the food items they consume into the application.
 - Searches for specific food items and accurately records the quantity or serving size.
- **Managing Diet Plans:** Has the option to create personalized diet plans based on goals and preferences.
 - Sets the start and end dates of diet plans, and monitors adherence, adjusting as necessary.
- **Reviewing Nutritional Information:** Reviews the nutritional information provided for food items, meals, and diet plans.
 - Interprets the nutritional values to make informed dietary decisions.
- **Following Recommendations & Guidance:** Considers and follows personalized recommendations and incorporates guidance into daily habits.
- **Monitoring Progress & Reporting:** Monitors progress towards goals and reviews generated reports and visualizations.
 - Reviews feedback and recommendations from the application and makes necessary adjustments.

7. User Flow

User Registration & Profile Setup

- The customer opens the app and signs up using their details.
- During registration, they provide age, weight, height, gender, and activity level.
- After registration, they can access their profile and update these details at any time.

Food Consumption Logging

- On the home screen/dashboard, the user sees an option to add food.
- They search for food items from the database, select an item, and input the quantity or serving size.
- The entry is saved in their daily log or diary.

Diet Plan Creation

- The user navigates to the “My Diet Plan” section and creates a new plan by choosing a goal (e.g., weight loss, maintenance).
- They select start and end dates and any preferred food types or restrictions. The plan is saved and tracked daily.

Nutritional Information Review

- The app displays the nutritional breakdown of each logged item (calories, protein, carbs, fats).
- Users can review total daily intake or a per-meal breakdown to understand what they are consuming.

Following Recommendations

- Based on input data and goals, the app provides personalized tips (e.g., increase protein intake), shown on the dashboard or in a notification area.

User Flow Loop

Figure 2: User Flow — Diet Analysis and Progress Loop

The flow is cyclical by design: once a user's diet is analyzed and suggestions are implemented, the system follows up and monitors progress. If the goal has not yet been achieved, the loop returns to the suggestions stage, keeping the guidance adaptive rather than one-time.

8. Execution Roadmap

- **Phase 1 — Requirement Analysis & Planning:** Identify business needs, target users, feature scope, and system architecture.
- **Phase 2 — UI/UX Design:** Design wireframes for registration, dashboard, food logging, and diet-plan screens.
- **Phase 3 — Data Modeling:** Design MongoDB collections and relationships for User, Food, Meal, DietPlan, and NutritionFact.
- **Phase 4 — Backend Development:** Build the Express.js REST API — controllers, models, routes, and middleware (including JWT authentication).
- **Phase 5 — Frontend Development:** Build the React.js interface for registration, food logging, diet plans, and reports, and connect it to the API.
- **Phase 6 — Nutrition Logic & Recommendation Engine:** Implement nutrition calculations and the recommendation logic (e.g., `utils/suggestNutrition.js`).
- **Phase 7 — Testing & Validation:** Perform functional, API, and integration testing across modules.
- **Phase 8 — Deployment & Documentation:** Finalize configuration, deploy the application, and prepare documentation.

9. System Requirements

Technology Stack

Frontend	HTML, CSS, JavaScript, React.js
Backend	Node.js, Express.js
Database	MongoDB (via Mongoose ODM)
Authentication	JSON Web Tokens (JWT)
Tooling	VS Code, Postman, Git/GitHub

Functional Requirements

- User registration, login, and profile management.
- Food search, selection, and consumption logging.
- Diet plan creation, monitoring, and updates.
- Nutritional analysis and recommendation generation.
- Report and progress visualization.

Non-Functional Requirements

- Scalability to support a growing food database and user base.
- Data accuracy in nutrition calculations.
- Security of user profile and health-related data.
- Reliability and easy maintainability of the codebase.

10. Data Architecture & Object Model

The data model is centered on five entities: User, Food, Meal, DietPlan, and NutritionFact.

ER Diagram

Figure 3: Entity-Relationship (ER) Diagram

- **User:** Includes attributes such as weight, height, gender, and activity level, in addition to standard profile fields.
- **Food:** Includes attributes for calories, carbohydrates, proteins, and fats to represent the nutritional information of each food item.
- **Meal:** Includes `userId` to associate a meal with a specific user, and `date` to record when the meal was logged.
- **DietPlan:** Includes `userId` to link the plan with a user, plus `startDate` and `endDate` to define its duration.
- **NutritionFact:** Includes foreign key attributes `mealId` and `dietPlanId`, establishing its relationship with the corresponding meal or diet plan.

Key Relationships

- A User creates Meals and follows a DietPlan.

- A User consumes Food, which is included in a Meal.
- A Meal has an associated NutritionFact, and is included within a DietPlan.
- A DietPlan recommends NutritionFacts based on the user's logged data and goals.

11. Application Structure (Backend)

The backend follows a modular Express.js structure, organized by responsibility:

- **controllers/**
 - SuggestedController.js — handles logic for creating and retrieving nutrition suggestions for users.
 - userController.js — manages user-related actions such as registration, login, and fetching user profiles.
- **db/**
 - config.js — sets up the MongoDB connection using `mongoose.connect()`.
- **middlewares/**
 - authMiddleware.js — verifies JWT tokens to protect routes and ensure only authenticated users can access certain endpoints.
- **models/**
 - Suggestion.js — Mongoose model defining the schema for storing nutrition suggestions.
 - User.js — Mongoose model defining the schema for user data, including email, password, and profile information.
- **routes/**
 - suggestionRoute.js — defines API endpoints for nutrition suggestions and links them to controller functions.
 - userRoute.js — defines user-related API endpoints (register, login) and connects them to user controller functions.
- **utils/**
 - suggestNutrition.js — contains the core logic that generates personalized nutrition suggestions based on user input.

Entry Point — server.js

- Initializes the Express server.
- Connects to MongoDB via db/config.js.

- Registers middleware (express.json(), cors, etc.).
- Registers all route files.
- Listens on a configured port (e.g., 8000).

12. Testing

- **Authentication Testing:** Verifying registration, login, and JWT-protected routes behave correctly.
- **API Endpoint Testing:** Validating each Express endpoint (users, food, meals, diet plans) with tools such as Postman.
- **Nutrition Calculation Testing:** Confirming calorie, carbohydrate, protein, and fat totals are computed correctly for meals and diet plans.
- **CRUD Testing:** Testing create, read, update, and delete operations for diet plans and food logs.
- **Database Integration Testing:** Confirming reads/writes against MongoDB collections behave as expected.
- **UI/UX Testing:** Checking usability of registration, logging, and dashboard screens across devices.

13. Maintenance and Troubleshooting

Maintenance Activities

- Keeping the food and nutrition database current and accurate.
- Monitoring API performance and uptime.
- Reviewing user feedback to refine the recommendation logic.
- Updating Node.js/React.js dependencies and addressing security advisories.

Troubleshooting

- Reviewing server logs to diagnose API errors.
- Checking MongoDB connection status for data access issues.
- Validating JWT tokens when authentication errors occur.
- Investigating data inconsistencies in nutrition calculations.
- Ensuring front-to-backend data payloads match strict specifications.

14. Learning Outcomes

- Practical experience building a full-stack application with the MERN stack.
- Understanding of RESTful API design using Express.js.
- Hands-on MongoDB schema design and Mongoose modeling.
- Implementing authentication and route protection with JWT.
- Experience designing recommendation logic for a health and wellness application.
- Practice translating architecture, ER, and user-flow diagrams into working code structure.

15. Output of the Project

The final system delivers a full-stack Nutrition Assistant application that allows users to register and manage a health profile, log food consumption against a nutrition database, create and monitor personalized diet plans, and view nutritional analysis and progress reports. Personalized recommendations, generated from the user's profile and logged data, help guide healthier choices over time.

16. Conclusion

The Nutrition Assistant demonstrates how a MERN stack application can bring together profile management, food and nutrition data, and personalized recommendation logic into a single, coherent tool for everyday diet management. By combining structured data modeling, a modular Express.js backend, and a guided user flow, the system gives users a practical way to track intake, follow diet plans, and work toward their health goals — while continuing to encourage consultation with qualified healthcare providers for personalized medical advice.